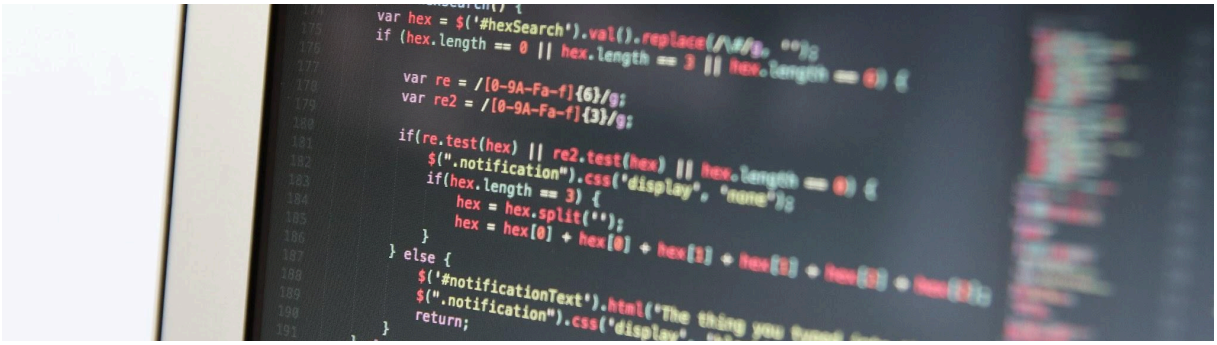

Status Completed

SOCIOMILE 2.0

Engineering Take-Home Coding Assignment (Fullstack)

Level	Medior – Senior
Type	Take-Home Assignment
Stack yang Disarankan	**Backend**: Golang • Database: MySQL • Cache (Opsional): Redis • API Style: REST API • Auth: JWT / Token-based **Frontend**: ReactJs / VueJs

---

Introduction

Sociomile adalah *platform omnichannel customer support* yang dirancang dengan pendekatan service-oriented dan scalable architecture.

Melalui tugas ini, kami ingin menilai:

- Cara berpikir engineering
- Kemampuan merancang sistem dan memodelkan data nyata
- Kualitas implementasi fullstack
- Kematangan dalam menulis *production code*, dan
- Pemahaman *lifecycle* dari platform customer support

Tugas ini tidak mengharuskan semua fitur selesai 100%, namun kami menilai prioritas, struktur, dan kualitas solusi.

Deskripsi Case Study

Sebuah perusahaan (tenant) menerima pesan dari customer melalui berbagai channel (misalnya *WhatsApp* atau *Instagram*). Setiap pesan yang masuk akan membentuk sebuah conversation di dalam sistem. Conversation tersebut dapat di-assign ke agent untuk dibalas.

Jika percakapan membutuhkan penanganan lebih lanjut, agent dapat melakukan eskalasi menjadi *ticket* untuk diproses secara internal hingga selesai.

Sistem harus mendukung multi-tenant (lebih dari satu perusahaan) dan memastikan data antar tenant terisolasi.

Fokus utama dari case ini adalah bagaimana Anda mendesain flow berikut:

Channel → Conversation → Assignment to Agent Reply → Escalate to Ticket

---

Functional Requirements

1. **Authentication & Authorization**
    a. Login menggunakan email + password
    b. Role: ***`admin`*** & ***`agent`***
    c. Proteksi endpoint berdasarkan role
    d. Menggunakan token (JWT atau sejenis)

2. **Channel → Conversation Flow**

   Sistem mensimulasikan channel eksternal (misal WhatsApp, Instagram, dll), tidak perlu benar-benar integrasi API eksternal — cukup di simulasikan via endpoint.

   Channel Message: Customer mengirim pesan melalui channel.

   API: `POST /channel/webhook`

   Payload (minimal):

   ```
   {
     "tenant_id": "...",
     "customer_external_id": "...",
     "message": "Hello"
   }
   ```

   Sistem harus:

   a. Mencari conversation existing
   b. Jika belum ada → buat conversation baru
   c. Simpan message ke conversation
   d. Conversation memiliki status (open / assigned / closed)

3. **Conversation Management**

   Conversation adalah entity utama sebelum menjadi ticket.

   Conversation Attributes (minimal)

   - **id**
   - **tenant_id**
   - **customer_id**
   - **status (open, assigned, closed)**
   - **assigned_agent_id (nullable)**

   Message Attributes (minimal)

- **id**
- **conversation_id**
- **sender_type (customer / agent)**
- **message**

API Minimal:

- Get Conversation List: `GET /conversations` dengan filter: status, assigned_agent, pagination
- Get Detail Conversation: `GET /conversations/{id}`
- Agent Reply: `POST /conversations/{id}/messages`

4. **Escalation to Ticket**

   Conversation bisa di eskalasi menjadi ticket. Escalation Rules:

   - Hanya agent yang bisa escalate
   - 1 conversation → maksimal 1 ticket
   - Hanya admin yang bisa ubah status ticket
   - Setelah escalate:
     - Conversation tetap ada
     - Ticket dibuat dan terhubung ke conversation

   Ticket Minimal Attributes

   - **id**
   - **tenant_id**
   - **conversation_id**
   - **title**
   - **description**
   - **status (open, in_progress, resolved, closed)**
   - **priority**
   - **assigned_agent_id**

API Minimal

- Escalate conversation → create ticket
- Update ticket status by admin
- List ticket

Technical Requirements

1. Database Design: Gunakan MySQL atau Database lain dengan relasi tabel yang jelas dan penggunaan index serta wajib menyertakan ***Schema SQL / migration***
2. Code Structure & Quality yang konsisten, gunakan error handling dan logging dasar

   Separation of concern (backend):

   - handler
   - service / usecase
   - repository / data access
3. Wajib mendukung multi-tenancy (data antar tenant terisolasi).
4. Gunakan *foreign key* dan *index* yang relevan (misalnya pada tenant_id, conversation_id, ticket_id, dll).
5. Documentation `**README.md**` minimal berisi:
   a. Cara menjalankan aplikasi
   b. Environment variables yang digunakan
   c. Daftar endpoint API
   d. Penjelasan singkat arsitektur
   e. Asumsi dan trade-off yang diambil
   f. Penjelasan pendekatan multi-tenancy
6. (*Nice to have*) menyertakan OpenAPI / Swagger.

Advanced Requirements (*nice to have*)

Implementasi salah satu atau lebih:

⭐ **Event / Async Processing**

Trigger event saat:

- Conversation di-assign ke agent
- Conversation di-escalate menjadi ticket
- Ticket dibuat

Event dapat:

- Disimpan ke table `events` / `activity_logs`
- Dipublish ke Redis / message queue
- Diproses oleh async worker

⭐ **Redis**

Gunakan Redis untuk salah satu atau lebih:

- Cache list conversation
- Cache list ticket
- Rate limiting endpoint webhook (simulasi channel)
- Simple pub/sub untuk notifikasi internal

⭐ **Docker**

Buat `docker-compose.yml` untuk menjalankan:

- Backend service
- Database (MySQL)
- Redis (jika digunakan)
- Optional: Queue (Kafka / RabbitMQ)

⭐ **Testing**

Implementasikan unit test untuk business logic utama. Fokus pada:

- Conversation lifecycle

- Assignment logic
- Escalation ke ticket (pastikan 1 conversation hanya bisa jadi 1 ticket)
- Authorization logic di service layer

Testing tidak perlu full coverage, tetapi menunjukkan pemahaman struktur layer dan separation of concern.

⚠️ Aturan Penggunaan AI

Tujuan tes ini adalah memahami cara berpikir dan proses kerja kamu, bukan hasil akhir.

⚠️ *Batasi penggunaan AI code generator atau Agent AI (ChatGPT, Copilot, Cursor, Kuro, Claude, dsb) untuk menulis langsung kode utama.*

✅ *Kamu boleh membaca dokumentasi resmi, artikel, atau sumber pembelajaran terbuka.*

Deliverables

Silakan kumpulkan dalam bentuk:

- Repository Git (*GitHub*)
- ***`README.md`***
- (Opsional) Screenshot atau demo link

📝 Notes

Tugas ini dirancang untuk merepresentasikan pekerjaan nyata yang akan Anda hadapi di Sociomile, bukan sekadar tes akademis.

Tidak diwajibkan untuk menyelesaikan seluruh fitur secara sempurna.

Kami lebih menghargai:

- Cara berpikir dan pengambilan keputusan teknis
- Struktur kode yang rapi dan mudah dikembangkan
- Kejelasan asumsi dan trade-off yang Anda ambil

Jika waktu terbatas, silakan:

- Fokus pada core requirement
- Jelaskan di README bagian mana yang belum sempat dikerjakan dan alasannya

Gunakan pendekatan dan teknologi yang paling Anda kuasai — kami tertarik melihat strength Anda sebagai engineer.

👉 Tidak ada jawaban "paling benar". Kami ingin melihat cara Anda membangun solusi.

Kami menantikan hasil karya Anda dan berharap dapat mendiskusikannya lebih lanjut pada tahap interview berikutnya.

Semoga proses ini juga menjadi pengalaman yang menyenangkan dan bermanfaat bagi Anda.

Good luck, and happy coding! 🚀